FINAL PROJECT REPORT

ON

CHESS BOARD(JAVA GUI APPLICATION)

Submitted by:

Jagpreet Singh
Keshav katoch
Mohit Chauhan
Ridham
UID : 23BCA10376
23BCA10016
23BCA10018
23BCA10767

Submitted to:

Ms Jyoti Dhiman

Final Project Report

CHESS BOARD (JAVA GUI APPLICATION)

1. Title

CHESS BOARD

---

2. Objective

The main objective of this project is to design and develop a Chess Game using Java Swing that allows users to play chess in an interactive graphical interface. The system focuses on providing an engaging and visually appealing experience with custom-designed game elements.

The system aims to:

- Provide a user-friendly and interactive chess board interface
- Enable smooth piece movement through mouse-based interactions
- Track and display turns between two players (White and Black)
- Maintain and display captured pieces during gameplay
- Offer a reset option to start a new game instantly

---

3. Problem Statement

Many traditional chess setups lack an interactive digital interface and do not provide visual tracking of moves and captured pieces, which can make the gameplay less engaging and harder to manage. There is a need for a system that can:

• Provide a visually organized chess board
• Allow players to move pieces easily through mouse interactions
• Track turns and enforce basic gameplay flow
• Display captured pieces clearly during the game

This project solves these problems by creating a GUI-based Chess Game using Java Swing, offering an interactive and user-friendly playing experience

---

4. Tools and Concepts Used

Technologies Used:

- **Language:** Java
- **GUI Framework:** Java Swing
- **IDE:** IntelliJ IDEA / Eclipse / NetBeans
- **Platform:** Desktop Application

Concepts Implemented:

- Object-Oriented Programming (OOP)
- Event Handling (ActionListener)
- GUI Design using Swing Components
- Arrays and Data Structures
- Conditional Logic
- Score Calculation Algorithm

---

5. System Design

The Modules of the System:

1. **User Interface (GUI):**
   o Displays the chessboard using Java Swing
   o Uses custom-painted components to render chess pieces
   o Provides an interactive interface for selecting and moving pieces
   o Includes status bar and control buttons (e.g., New Game)

2. **Board Management Module:**
   o Maintains the chessboard using a 2D array
   o Stores positions of all pieces
   o Initializes the board with standard chess setup
   o Updates board state after each move

3. **Move Handling Module:**
   o Handles user clicks for selecting and moving pieces
   o Highlights possible move positions
   o Prevents invalid actions like selecting opponent's piece

o Switches turns between White and Black

4Capture Management Module:
o Tracks captured pieces for both players
o Stores captured pieces in separate lists
o Updates the display panel for captured pieces dynamically

5. Game Control Module:
o Manages turn-based gameplay
o Resets the game to initial state when required
o Updates status messages (e.g., “White’s Turn”, “Black’s Tu

Working Flow:
• Application starts
• Chess board is initialized with pieces in starting positions
• White player selects a piece using mouse click
• Possible moves are highlighted
• Player selects a destination square
• Move is executed and board updates
• If an opponent’s piece is present, it is captured and displayed in the side panel
• Turn switches to the other player (White ↔ Black)
• Process repeats for each move
• User can click “New Game” to reset the board

---

6. Flowchart / Workflow

Start

↓

Initialize Chess Board

↓

Display Board on GUI

↓

Player Selects a Piece

↓

Highlight Possible Moves

↓

Player Selects Destination

↓

Valid Move?

↓ Yes → Execute Move & Update Board

↓ No → Cancel Selection

↓

Capture Opponent Piece (if present)

↓

Switch Turn (White ↔ Black)

↓

New Game Selected?

↓ Yes → Reset Board

↓ No → Continue Game

↓

End

7. Code Snippet / Module Explanation:

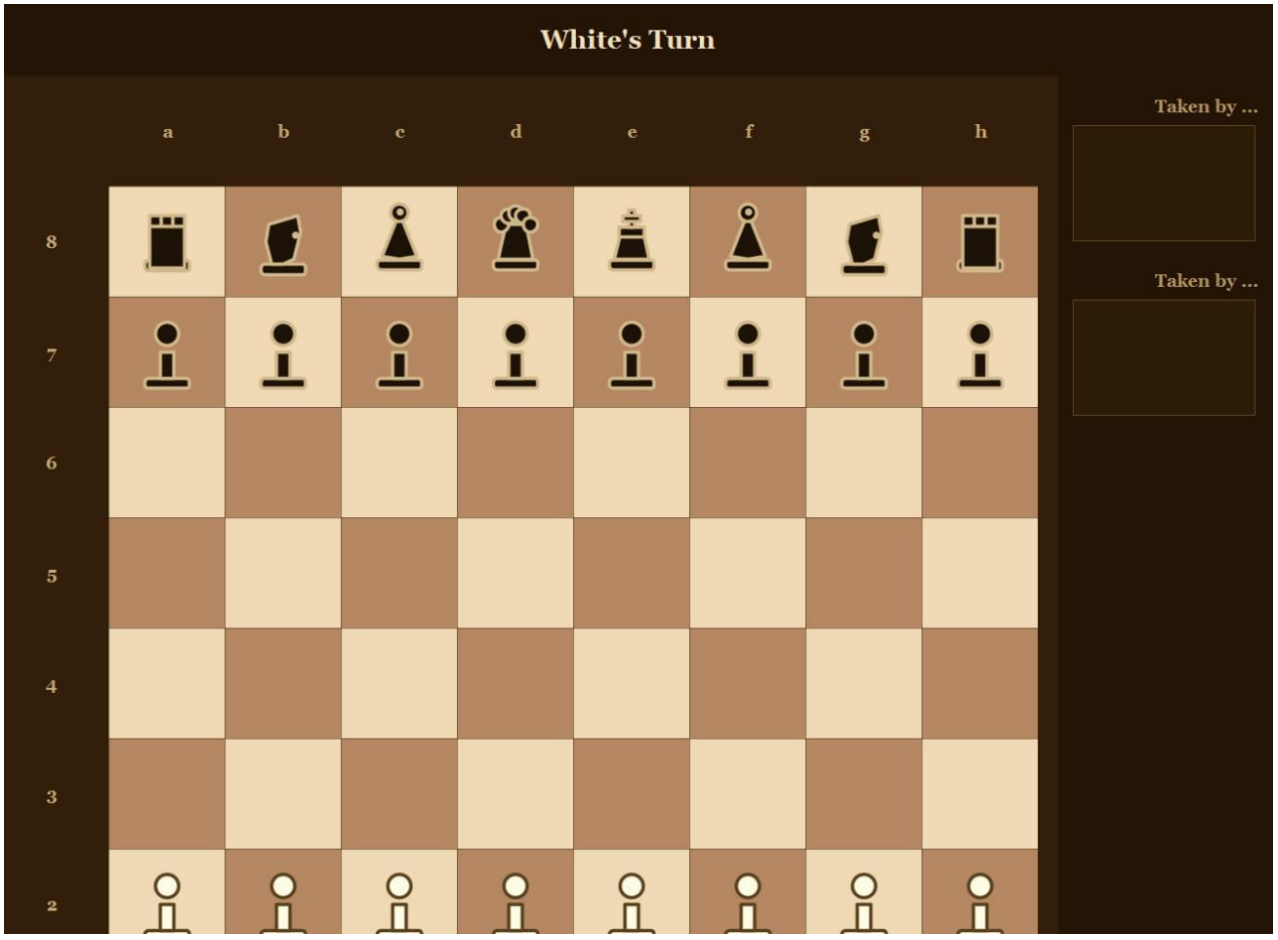
```java
import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.event.*;
import java.awt.geom.*;
import java.util.ArrayList;
import java.util.List;

public class App {

    private JFrame frame;  13 usages
    private ChessSquare[][] squares = new ChessSquare[8][8];  15 usages
    private String[][] board = new String[8][8];  13 usages
    private int selectedRow = -1, selectedCol = -1;  10 usages
    private boolean whiteTurn = true;  6 usages
    private JLabel statusLabel;  6 usages
    private JPanel capturedWhitePanel, capturedBlackPanel;  6 usages
    private List<String> capturedByWhite = new ArrayList<>();  3 usages
    private List<String> capturedByBlack = new ArrayList<>();  3 usages

    private static final int SQ = 80; // square size in pixels  8 usages

    private static final Color LIGHT_SQ  = new Color( r: 240,  g: 217,  b: 181);  1 usage
    private static final Color DARK_SQ   = new Color( r: 181,  g: 136,  b: 99);  1 usage
```

```java
public class App {
    static class ChessSquare extends JPanel {  7 usages  1 inheritor
        String piece;  7 usages
        Color baseColor;  2 usages
        boolean highlighted, selected;  4 usages

        ChessSquare(Color base) {  2 usages
            this.baseColor = base;
            setPreferredSize(new Dimension(SQ, SQ));
            setMinimumSize(new Dimension(SQ, SQ));
            setMaximumSize(new Dimension(SQ, SQ));
            setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        }

        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
            int w = getWidth(), h = getHeight();

            g2.setColor(baseColor);
            g2.fillRect( x: 0,  y: 0, w, h);

            if (selected) {
```

```java
private void drawPiece(Graphics2D g2, String p, int w, int h) {  1 usage
    boolean white = Character.isUpperCase(p.charAt(0));
    int cx = w / 2, cy = h / 2;
    int sz = (int)(Math.min(w, h) * 0.82);

    Color fill   = white ? new Color( r: 255,  g: 252,  b: 230) : new Color( r: 28,  g: 18,  b: 8);
    Color stroke = white ? new Color( r: 90,  g: 65,  b: 30)    : new Color( r: 210,  g: 185,  b: 140);

    g2.setStroke(new BasicStroke( width: sz * 0.038f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));

    switch (Character.toUpperCase(p.charAt(0))) {
        case 'P' -> drawPawn(g2, cx, cy, sz, fill, stroke);
        case 'R' -> drawRook(g2, cx, cy, sz, fill, stroke);
        case 'N' -> drawKnight(g2, cx, cy, sz, fill, stroke);
        case 'B' -> drawBishop(g2, cx, cy, sz, fill, stroke);
        case 'Q' -> drawQueen(g2, cx, cy, sz, fill, stroke);
        case 'K' -> drawKing(g2, cx, cy, sz, fill, stroke);
    }
```

```java
private JPanel buildSidePanel() {  1 usage
    JPanel side = new JPanel();
    side.setLayout(new BoxLayout(side, BoxLayout.Y_AXIS));
    side.setBackground(PANEL_BG);
    side.setBorder(new EmptyBorder( top: 14,  left: 10,  bottom: 14,  right: 16));
    side.setPreferredSize(new Dimension( width: 160,  height: 0));

    side.add(sectionTitle( t: "Taken by White"));
    side.add(Box.createVerticalStrut( height: 6));
    capturedWhitePanel = captureBox();
    side.add(capturedWhitePanel);
    side.add(Box.createVerticalStrut( height: 20));
    side.add(sectionTitle( t: "Taken by Black"));
    side.add(Box.createVerticalStrut( height: 6));
    capturedBlackPanel = captureBox();
    side.add(capturedBlackPanel);
    side.add(Box.createVerticalGlue());

    JButton reset = new JButton( text: "New Game");
    reset.setFont(new Font( name: "Georgia", Font.BOLD,  size: 13));
    reset.setBackground(new Color( r: 160,  g: 100,  b: 40));
```

OUTPUT:



8. Conclusion

The Chess Game successfully demonstrates the implementation of a GUI-based application using Java Swing. It provides an interactive and user-friendly platform for two players to play chess with visually appealing, custom-designed pieces.

The project highlights the use of event-driven programming, object-oriented programming (OOP) concepts, and advanced GUI design, making it a practical example of desktop application development.

This system enhances the gaming experience by providing smooth piece movement, turn tracking, and capture management, while eliminating the need for a physical chessboard.

9. Limitations

No Advanced Game Rules Implementation:
Basic movement is allowed, but advanced rules like check, checkmate, castling, and en passant are not implemented

• **No AI Opponent:**
The game supports only two-player mode; there is no computer-based opponent

• **No Move Validation Logic:**
The system does not fully restrict illegal moves according to official chess rules

• **No Game History Tracking:**
Previous moves are not recorded or displayed

• **Basic UI Enhancements:**
Although visually appealing, the interface lacks animations and advanced interactivity

Future Scope

- **Implement Complete Chess Rules:**
  Add advanced rules like check, checkmate, castling, en passant, and pawn promotion
- **Add AI Opponent:**
  Develop a computer-based opponent with different difficulty levels
- **Move Validation System:**
  Restrict moves strictly according to official chess rules
- **Game History & Undo Feature:**
  Store moves and allow players to undo/redo moves
- **Multiplayer Mode:**
  Enable online or local multiplayer functionality
- **Enhanced UI & Animations:**
  Improve visuals with animations and smoother interactions (can use JavaFX